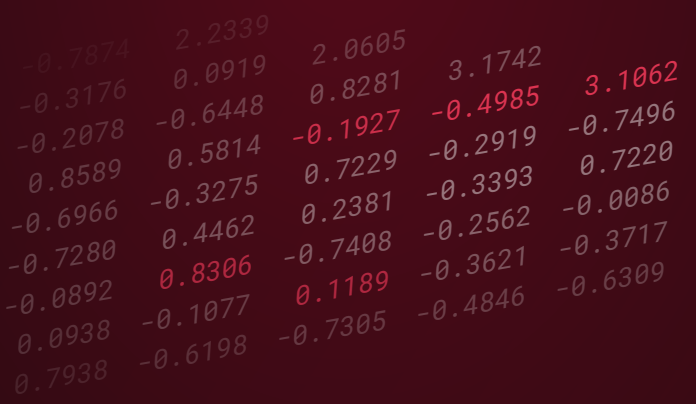
SPARSE ACCELERATED RAPID TRIANGULAR ANALYSER

SPARTA

USER GUIDE



Supernodal multifrontal LDL^T and LU for large sparse systems, with every numeric kernel on the GPU.

LDL^T AND LU
supernodal multifrontal, GPU-resident

REAL AND COMPLEX
single precision, refined to double

NESTED DISSECTION
SCALPEL, a parallel ordering

RUST, PYTHON, C/C++
MATLAB and Julia too

ROBERT FENNIS · MILAN ROTHER

CONTENTS

1 INTRODUCTION 4
1.1 What SPARTA does 4
1.2 How a direct solver works 4
1.3 Two factorization methods 5
1.4 Precision in one paragraph 5
1.5 What is supported 5
1.6 Key terms 7
2 INSTALLATION 8
2.1 Requirements per operating system 8
2.2 Packages you need 8
2.3 Installing the packages 8
2.4 Installing SPARTA 9
2.5 Checking the installation 10
2.6 A fast CPU backend on Linux and Windows 10
3 QUICK USE 12
3.1 How the matrix is passed 12
3.2 Solving in Rust 13
3.3 Solving in Python 14
3.4 Solving again 14
3.5 Freeing memory 15
4 MATLAB AND JULIA 16
4.1 MATLAB 16
4.2 Julia 17
4.3 Both bindings 17
5 SETTINGS AND CONFIGURATION 19
5.1 How settings work 19
5.2 Telling SPARTA about your matrix 19
5.3 Configuring in Rust 20
5.4 Configuring in Python 20
5.5 Presets 21
5.6 Tuning to your computer 22
5.7 All settings 24
5.8 Solve settings (Python only) 26
5.9 Ordering settings 26
5.10 Front settings 27
5.11 Numerical settings 27
5.12 CPU backend settings 28
5.13 GPU settings (advanced) 28
5.14 Environment variables 29
6 PRECISION 30
6.1 Why answers are not exact 30
6.2 The options at a glance 30
6.3 What to do: step by step 31
6.4 Checking the factorization 31
6.5 Getting more accuracy from the GPU 31
6.6 Solving on the CPU in double precision 33
6.7 CPU and GPU together 34

6.8 When refinement does not converge . . . 35
6.9 Checking the final answer . . . 35

CHAPTER 1

INTRODUCTION

This guide takes you from installing SPARTA to solving your own systems accurately. It assumes you can program in Rust or Python, but not that you know how a sparse direct solver works: the ideas are explained as they come up.

1.1 WHAT SPARTA DOES

SPARTA solves systems of linear equations,

$$Ax = b,$$

where A is a large square matrix, b is a known vector (the **right-hand side**) and x is the unknown vector you want. Such systems come out of simulations: a finite-element model of an antenna, a circuit or a structure produces one matrix per frequency or time step.

These matrices are **sparse**: nearly all their entries are zero. A matrix with a million rows might have only ten or twenty nonzeros per row. SPARTA stores and works on the nonzeros only, which is what makes such large systems solvable at all.

SPARTA is a **direct** solver. It computes the answer in a fixed number of steps, as opposed to an iterative solver, which improves a guess until it is good enough and may converge slowly or not at all on difficult systems. The price of a direct solver is memory and work up front; SPARTA pays that price on the GPU of an Apple Silicon Mac, where it is paid quickly.

1.2 HOW A DIRECT SOLVER WORKS

A direct solver does what you would do by hand with Gaussian elimination, but records the result so it can be reused. It splits the work into three steps.

1. **Analyze.** Look only at *where* the nonzeros are (the **pattern**), not at their values. Choose an order in which to eliminate the unknowns, and plan the memory and the computation. This runs on the CPU.
2. **Factor.** Use the values to split A into simpler matrices, its **factors**, whose product is A. This is the expensive step, and it runs on the GPU.
3. **Solve.** Use the factors to find x for a given b. This is cheap compared with factoring.

The split pays off because each step can be repeated on its own:

- **A new right-hand side** needs only a new solve.
- **New values in the same pattern**, such as the next frequency of a sweep, need a new factorization, but reuse the analysis and the memory it reserved.
- **A new pattern** needs a new analysis. In Python, `factorize` runs it for you when the matrix it is given has a different pattern.

Two things decide how much work and memory a factorization takes.

Fill-in. Eliminating an unknown creates new nonzeros in the factors where A had zeros. The factors are therefore denser than A itself, often by ten times or more.

Ordering. How much fill-in appears depends heavily on the order in which the unknowns are eliminated. Finding a good order is the main job of the analysis. SPARTA uses **nested dissection**: it repeatedly cuts the problem into two halves joined by a small **separator**, and eliminates the halves before the separator.

SPARTA gathers the work into dense blocks called **fronts**, because dense blocks are what a GPU computes fastest. A group of columns handled together in one front is a **supernode**.

1.3 TWO FACTORIZATION METHODS

The kind of matrix decides which factorization SPARTA can use.

METHOD	USED FOR	RELATIVE COST
LDL^T	**Symmetric** matrices, real or complex: A equals its own transpose, $A = A^T$. Only one triangle needs storing.	1×
LU	**Every other square matrix**: unsymmetric, only the pattern symmetric, or Hermitian (A equal to its conjugate transpose).	about 2×

> WARNING
>
> **Symmetric means equal to the transpose, not the conjugate transpose.** A complex Hermitian matrix is solved with LU. A symmetric pattern with unsymmetric values also needs LU.

While factoring, each step divides by one entry, the **pivot**. A pivot that is nearly zero amplifies rounding errors. Both methods therefore reorder rows or use small 2×2 blocks locally to avoid small pivots, which is called **pivoting**.

1.4 PRECISION IN ONE PARAGRAPH

The GPU computes in **single precision**, which holds about seven significant digits. That makes it fast, but on a typical simulation matrix the answer comes out with a **relative residual** of about 10^{-5}, where the residual $b - Ax$ measures how well x satisfies the equations. SPARTA can recover double-precision accuracy by **iterative refinement**, or compute entirely in double precision on the CPU. Section 6 explains exactly how.

Matrices may be real or complex, and SPARTA follows the type of the values you pass: a real matrix is factored with real arithmetic (float32 on the GPU, float64 on the CPU), a complex one with complex arithmetic (complex64 or complex128).

1.5 WHAT IS SUPPORTED

	SUPPORT
GPU	Apple Silicon (M1 or later) through Metal. AMD and NVIDIA GPUs on Windows and Linux through Vulkan, in alpha: correct, not yet tuned.
CPU	A double-precision backend on any 64-bit machine, used when you ask for it.
Languages	Rust and Python, covered in this guide, and MATLAB and Julia (Section 4). A C and C++ interface exists as well; see `crates/sparta-capi/README.md`.
Matrices	Square, real or complex, stored in compressed sparse column form (Rust) or any SciPy sparse format (Python). Real matrices are factored with real arithmetic.

	SUPPORT
Ordering	SCALPEL, SPARTA's own parallel nested dissection.

1.6 KEY TERMS

TERM	MEANING
Sparse matrix	A matrix whose entries are almost all zero, stored as its nonzeros only.
Pattern	The positions of the nonzeros, regardless of their values.
Right-hand side	The known vector b in $Ax = b$.
Factorization	Splitting A into factors that make solving easy.
Fill-in	Nonzeros the factors have where A had zeros.
Ordering	The order in which unknowns are eliminated. Decides the fill-in.
Front	A dense block of the factorization, computed as one unit.
Supernode	A group of columns factored together in one front.
Pivot	The entry divided by at each elimination step.
Residual	$b - Ax$. Its size relative to b shows how well x solves the system.
Refinement	Correcting an answer using its residual, to gain accuracy.
CSC	Compressed sparse column: the storage format SPARTA reads (Section 3.1).

CHAPTER 2

INSTALLATION

2.1 REQUIREMENTS PER OPERATING SYSTEM

macOS. An Apple Silicon Mac (M1 or later) running macOS 13 or later runs everything, including the GPU. An Intel Mac runs the analysis and the double-precision CPU backend, but has no GPU.

Windows. Any 64-bit machine runs the analysis and the CPU backend. AMD and NVIDIA GPUs run through the Vulkan backend, which is in alpha: tested on an NVIDIA RTX 3080 and an AMD Radeon integrated GPU, correct but not yet tuned for speed. A normal install includes it and needs nothing extra; running it needs a GPU driver with Vulkan 1.3. Without one, `gpu_available()` is false and `force_cpu=True` selects the CPU backend. For a fast CPU backend, install a BLAS library (Section 2.6).

Linux. The same as Windows: the analysis, the CPU backend and the alpha Vulkan backend. Not yet tested on Linux.

2.2 PACKAGES YOU NEED

PACKAGE	WHAT IT IS FOR
Rust toolchain (`rustc`, `cargo`)	Compiling SPARTA. Needed for every install route except a prebuilt Python wheel.
Xcode command line tools	macOS only: compiling the Metal bridge.
git	Downloading SPARTA from GitHub.
Python 3.9 or later, with pip	The Python package.
NumPy and SciPy	Used by the Python package. pip installs them automatically.
OpenBLAS or Intel MKL	Optional, Linux and Windows only: makes the CPU backend fast.

2.3 INSTALLING THE PACKAGES

macOS

```
# Metal bridge tools and git
xcode-select --install
# Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

For Python, use the installation you already run your simulations with (python.org, Homebrew or conda).

Linux (Debian, Ubuntu)

```
sudo apt install build-essential git python3 python3-pip python3-venv
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

On Fedora or RHEL, the first line is `sudo dnf install gcc git python3 python3-pip`.

Windows

1. Rust: download and run `rustup-init.exe` from https://rustup.rs. When it asks, let it install the Visual Studio C++ Build Tools, which Rust needs to link.
2. git: install from https://git-scm.com/download/win.
3. Python: install from https://www.python.org/downloads/.

Check that everything is found by opening a new terminal and running:

```
cargo --version
git --version
python3 --version     # "python --version" on Windows
```

2.4 INSTALLING SPARTA

SPARTA is not published on crates.io or PyPI. Each language has the same three routes:

1. **You have access to the GitHub repository.**
2. **You have a copy of the source, but no GitHub access.**
3. **You have prebuilt binaries**, so nothing needs compiling.

Routes 1 and 2 compile SPARTA, which takes a few minutes the first time.

2.4.1 RUST

The package is called `sparta-solver` and is imported as `sparta`.

1. With GitHub access. Add to your project's `Cargo.toml`:

```
[dependencies]
sparta-solver = { git = "ssh://git@github.com/FennisRobert/SPARTA.git" }
```

This needs an SSH key registered with your GitHub account. Cargo fetches private repositories most reliably through your own git, so also add this to `~/.cargo/config.toml`:

```
[net]
git-fetch-with-cli = true
```

2. From a copy of the source. Point Cargo at the folder that contains SPARTA's `Cargo.toml`:

```
[dependencies]
sparta-solver = { path = "../SPARTA" }
```

3. Prebuilt binaries. Not applicable: Cargo always compiles Rust libraries from source. If you received a source archive, unpack it and use route 2.

2.4.2 PYTHON

Install into the Python environment you run your simulations in. The package is imported as `sparta`. Routes 1 and 2 need the Rust toolchain installed first, because pip compiles SPARTA's Rust code during the install.

1. With GitHub access.

```
git clone git@github.com:FennisRobert/SPARTA.git
cd SPARTA
pip install .
```

Or in one line, without keeping a copy of the source:

```
pip install "git+ssh://git@github.com/FennisRobert/SPARTA.git"
```

2. From a copy of the source. Unpack it, then install from its top folder, the one containing `pyproject.toml`:

```
cd path/to/SPARTA
pip install .
```

3. From a prebuilt wheel. A wheel is a ready-compiled package that needs no Rust and no compiler. One wheel works for every Python from 3.9 up, on Apple Silicon only:

```
pip install sparta_solver-0.2.2-cp39-abi3-macosx_11_0_arm64.whl
```

2.5 CHECKING THE INSTALLATION

Rust: build the example in Section 3.2 with `cargo run --release`. It prints the solution of a small system.

Python:

```
import sparta
print(sparta.__version__)
print(sparta.gpu_available())   # True on an Apple Silicon Mac
```

If `gpu_available()` prints `False` on an Apple Silicon Mac, the process cannot see the GPU. This happens in some sandboxed or remote sessions; run from a normal Terminal window.

2.6 A FAST CPU BACKEND ON LINUX AND WINDOWS

This only matters if you use the double-precision CPU backend off a Mac. That backend hands its large dense operations to a BLAS library, chosen when SPARTA is compiled. Without one it still gives correct answers, just much more slowly. On a Mac, Apple's Accelerate is always used and nothing needs doing.

Set two environment variables while installing:

VARIABLE	MEANING
`SPARTA_BLAS_LIB`	Library to link, without the `lib` prefix: `openblas`, or `mkl_rt` for Intel MKL.
`SPARTA_BLAS_DIR`	Folder containing it, if it is not on the default search path.

```
# OpenBLAS
sudo apt install libopenblas-dev
SPARTA_BLAS_LIB=openblas pip install .          # or: cargo build --release

# Intel MKL (preferred on x86 processors)
source /opt/intel/oneapi/setvars.sh
SPARTA_BLAS_DIR=$MKLROOT/lib SPARTA_BLAS_LIB=mkl_rt pip install .
```

On Windows, install the `mkl` and `mkl-devel` packages from PyPI into a virtual environment, here at `C:\venv`, and point the build at its `Library` folder:

```
SPARTA_BLAS_DIR='C:\venv\Library\lib' SPARTA_BLAS_LIB=mkl_rt pip install .
```

`C:\venv\Library\bin` must be on `PATH` whenever SPARTA runs.

MKL is preferred on x86 because SPARTA spreads its own work across threads and needs to control BLAS threading per thread, which MKL allows and OpenBLAS does not.

CHAPTER 3

QUICK USE

This chapter solves a small system with every setting left at its default. The next chapter explains the settings.

3.1 HOW THE MATRIX IS PASSED

Python takes any SciPy sparse matrix and converts it for you.

Rust takes the matrix in **compressed sparse column** (CSC) form: three arrays that list the nonzeros column by column.

ARRAY	CONTENTS
`col_ptr`	$n + 1$ offsets. The nonzeros of column j are entries `col_ptr[j]` up to, but not including, `col_ptr[j + 1]` of the other two arrays.
`row_idx`	The row of each nonzero, counting from zero, ascending and without repeats within a column.
`values`	The value of each nonzero, in the same order.

For example, the symmetric matrix

$$A = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

has this lower triangle in CSC form:

```
// Column 0 is entries 0-1, column 1 is entries 2-3, column 2 is entry 4.
let col_ptr = [0, 2, 4, 5];
let row_idx = [0, 1, 1, 2, 2];
let values  = [4.0, 1.0, 3.0, 1.0, 2.0];
```

For a symmetric matrix, pass only the lower triangle: the entries on and below the diagonal. For any other matrix, pass every nonzero.

3.2 SOLVING IN RUST

```rust
use sparta::{Analysis, Config, Solver};

fn main() -> sparta::Result<()> {
    // A = [[4, 1, 0], [1, 3, 1], [0, 1, 2]], symmetric: lower triangle only.
    let col_ptr = [0, 2, 4, 5];
    let row_idx = [0, 1, 1, 2, 2];
    let values: [f32; 5] = [4.0, 1.0, 3.0, 1.0, 2.0];
    let b: [f32; 3] = [1.0, 2.0, 3.0];

    // 1. Analyze the pattern.
    let analysis = Analysis::ldlt(3, &col_ptr, &row_idx, &Config::default())?;
    let mut solver = Solver::new(analysis)?;
    // 2. Factor the values.
    solver.factor_values(&values)?;
    // 3. Solve for x: one right-hand side.
    let x = solver.solve_values(&b, 1)?;
    println!("{x:?}");
    // 4. Free the GPU memory.
    solver.destroy();
    Ok(())
}
```

`Analysis::ldlt` is for symmetric matrices. For any other matrix, use `Analysis::lu` and pass the whole matrix.

`factor_values` and `solve_values` take `f32`, `f64`, `Complex32` or `Complex64`, and the type decides the arithmetic. The example above is a real matrix, so it passes `f32`. For a complex matrix, pass `Complex32` values and a `Complex32` right-hand side instead:

```rust
use sparta::Complex32;
let values = [4.0, 1.0, 3.0, 1.0, 2.0].map(|v| Complex32::new(v, 0.5));
let b = [1.0, 2.0, 3.0].map(|v| Complex32::new(v, 0.0));
solver.factor_values(&values)?;
let x = solver.solve_values(&b, 1)?;   // x is Complex32
```

- **Which types where.** The GPU takes `f32` and `Complex32`. The CPU backend (`force_cpu`, Section 6.6) takes all four and computes in the precision given.
- **Speed.** On the CPU a real matrix factors about twice as fast as the same matrix stored as complex. On the GPU, real matrices run through the complex kernels for now, so they are no faster there yet.
- **Right-hand sides.** A right-hand side must have the precision of the factors. A complex one may be solved against real factors; a real one against complex factors is refused, since its solution is complex.

`factor` and `solve` remain for `Complex32` values.

3.3 SOLVING IN PYTHON

```python
import numpy as np
from scipy.sparse import csc_matrix
import sparta

A = csc_matrix([[4.0, 1.0, 0.0],
                [1.0, 3.0, 1.0],
                [0.0, 1.0, 2.0]])
b = np.array([1.0, 2.0, 3.0])

solver = sparta.analyze(A)   # 1. analyze
solver.factorize()           # 2. factor
x = solver.solve(b)          # 3. solve
solver.destroy()             # 4. free
```

Python looks at the matrix and chooses the method itself: LDL^T if it is symmetric, LU otherwise. It also follows the dtype. This A is real, so it is factored with real arithmetic and `x` is a real array. A complex A, or a complex `b`, gives a complex `x`:

```python
A = csc_matrix([[4.0 + 0.5j, 1.0, 0.0],
                [1.0, 3.0 + 0.5j, 1.0],
                [0.0, 1.0, 2.0 + 0.5j]])   # complex: complex arithmetic
x = sparta.analyze(A).factorize().solve(b)  # x is complex128
```

`solver.compute_dtype` reports the arithmetic: float32 or complex64 on the GPU. Python also refines the answer to double-precision accuracy by default (Section 6).

3.4 SOLVING AGAIN

The three steps can be repeated separately.

A new right-hand side: call `solve` again. Several at once go faster together:

```rust
// Rust: the columns end to end; column j is b[j*n..(j+1)*n].
let x = solver.solve_values(&b, nrhs)?;
```

```python
# Python: B has shape (n, nrhs), one right-hand side per column.
X = solver.solve_many(B)
```

New values with the same pattern, such as the next frequency: factor again. The analysis and the memory are reused.

```rust
// Rust: the values in the same CSC order as the first time. They may change
// type, say from real to complex; the arithmetic follows.
solver.factor_values(&new_values)?;
```

```python
# Python: same pattern, new values.
solver.factorize(matrix=A_next)
```

A new pattern: start again from the analysis. In Python, `factorize` does this itself: given a matrix with a different pattern, or a different size, it analyzes again with the same settings and then factors. `solver.reanalyses` counts how often that happened. Each one costs a full analysis and new factor memory, so a count that keeps rising in a loop means the pattern is not stable between calls, for example because explicit zeros come and go.

3.5 FREEING MEMORY

A factorization of a large system can hold more than a gigabyte of GPU memory until its solver is freed. Call `destroy()` when you are done, as in the examples above. In Rust, dropping the solver also frees it. In Python the memory otherwise stays claimed for as long as anything refers to the solver, so prefer `destroy()` or a `with` block:

```
with sparta.analyze(A) as solver:
    solver.factorize()
    x = solver.solve(b)
```

CHAPTER 4

MATLAB AND JULIA

SPARTA can also be used from MATLAB and Julia. Both go through SPARTA's C library, so they share its abilities and its limits:

- They solve on the GPU only. The CPU backend (`force_cpu`) and the tuning search (Section 5.6) are not available.
- Refinement in double precision is done in the binding itself, more simply than in Python: it stops when the residual is small enough or stops improving.

Both are built from the source:

1. Build the C library from the repository's top folder. This needs Rust and nothing else.

```
cargo build --release -p sparta-capi
```

2. Use the files in `bindings/matlab` or `bindings/julia`, as below.

The settings have the same names and meanings as in Section 5.7. The binding-specific ones are as follows:

- `kind` chooses between LDL^T and LU, as `mtype` does in Python.
- `preset` picks the starting point (Section 5.5).
- `refinement_steps` and `refinement_rtol` control refinement, as they do in Python.

4.1 MATLAB

Not yet tested in MATLAB.

- The binding was written without access to MATLAB.
- Its C part has been run against the library through a stand-in for MATLAB, but the MATLAB class itself has not been run.
- Run `test_sparta` first, and report a failure as a GitHub issue.

Requirements:

- MATLAB R2018a or later.
- A C compiler that `mex -setup C` accepts. On Windows that is Visual Studio.
- On Apple Silicon, the native MATLAB, R2023b or later.

Building: in MATLAB, from `bindings/matlab`:

```
build_sparta_mex     % compiles the gateway against target/release
test_sparta          % checks the binding end to end
```

Then add that folder to the MATLAB path.

Solving: `SpartaSolver` works like MATLAB's own `decomposition`:

```
S = SpartaSolver(A);                    % analyze and factor
x = S \ b;                              % solve; b may have several columns

S = SpartaSolver(A, 'preset', 'sweep', 'nemin', 32);
for f = frequencies
    factorize(S, K - f^2 * M);          % same pattern: plan and memory reused
    x = S \ b;
end
clear S                                 % frees the GPU memory
```

Settings go in as name-value pairs. Errors carry identifiers such as `sparta:pivot` for a singular matrix. `help SpartaSolver` and `bindings/matlab/README.md` list the rest.

4.2 JULIA

Requirements: Julia 1.9 or later. The package calls the library directly, so it needs no compiler.

Installing: add the package to an environment, then run its tests:

```
using Pkg
Pkg.develop(path = "bindings/julia")
Pkg.test("SPARTA")
```

It loads the library from `target/release`. To use a library elsewhere, set the environment variable `SPARTA_LIB` to its path.

Solving:

```
using SPARTA, SparseArrays

F = SPARTA.Solver(A)                    # analyze and factor
x = F \ b                               # solve; b may have several columns

F = SPARTA.Solver(A; preset = :sweep, nemin = 32)
for f in frequencies
    factorize!(F, K - f^2 * M)          # same pattern: plan and memory reused
    x = F \ b
end
close(F)                                # frees the GPU memory now
```

Settings are keyword arguments. A failed call throws `SpartaError`, whose `kind` is one of the following:

- `:pivot`: the matrix is singular or too badly scaled.
- `:unsupported`: no GPU is available.
- `:device`: the GPU failed.
- `:invalid`: an input or setting was rejected.

Several tasks may share one solver, because its calls are serialized. `bindings/julia/README.md` lists the rest.

4.3 BOTH BINDINGS

- **Pass the whole matrix**, both triangles, even when it is symmetric. Refinement multiplies by it.

- **Changing pattern.** A later matrix may lack some entries of the first, for example because MATLAB drops entries that compute to exactly zero. The plan is then kept, with zeros in their place. A matrix with an entry the plan does not have is analyzed again.
- **Precision.** Answers come back in double precision. With `refinement_steps` set to 0 they carry the GPU's single-precision accuracy.
- **Real or complex.** Julia follows the element type: a real matrix is factored with real arithmetic and, with a real `b`, returns `Float64`; a complex one returns `ComplexF64`. MATLAB factors every matrix as complex for now and returns `complex double`.

CHAPTER 5

SETTINGS AND CONFIGURATION

The defaults are measured and suit most systems. This chapter explains every setting, and how to set them from Rust and Python.

5.1 HOW SETTINGS WORK

Settings belong to one of two moments:

- **Analysis settings** shape the plan: the ordering, the size of fronts, the GPU kernel shapes. They are fixed once the matrix is analyzed. To change one, analyze again.
- **Solve settings** (Python only) control refinement and checking. They can be changed for every factorization.

Every setting has the same name in Rust and Python, with one exception: in Rust, the four SCALPEL settings live inside a nested `scalpel` field.

5.2 TELLING SPARTA ABOUT YOUR MATRIX

5.2.1 IN RUST

You choose the method by the constructor you call, and store the matrix to match. Rust does not check symmetry, because it only ever sees one triangle.

CONSTRUCTOR	PASS	FOR
`Analysis::ldlt(n, &col_ptr, &row_idx, &config)`	lower triangle	symmetric matrices
`Analysis::lu(n, &col_ptr, &row_idx, &config)`	whole matrix	every other square matrix

5.2.2 IN PYTHON

By default Python inspects the matrix and picks LDL^T if it equals its transpose, LU otherwise. Three arguments to `sparta.analyze` change that:

ARGUMENT	DEFAULT	EFFECT
`mtype`	`MatrixType.AUTO`	Declares what kind of matrix this is. SPARTA checks the declaration: if you declare a matrix symmetric and it is not, `analyze` raises `ValueError`.
`path`	`Path.AUTO`	Forces a method: `Path.LDLT` or `Path.LU`.
`triangle`	`"full"`	`"lower"` if you pass only the lower triangle of a symmetric matrix. LU and Hermitian input need `"full"`.

The symmetry check allows a small absolute difference between mirrored entries, `sparta.EPS`, which starts at `1e-10`. Set it before calling `analyze`; `sparta.EPS = 0` demands exact symmetry. With a larger tolerance, LDL^T uses the lower triangle as it is, not an average of the two triangles.

5.3 CONFIGURING IN RUST

All analysis settings live in one `Config` struct, which you pass to `Analysis::ldlt` or `Analysis::lu`. Start from the defaults or a preset and change only what you need:

```
use sparta::{Config, OrderingMethod, Preset};

let mut config = Config {
    ordering: OrderingMethod::Scalpel,
    nemin: 32,
    ..Config::default()
};
config.scalpel.wide_cycles = 6;          // SCALPEL settings are nested

let config = Preset::Sweep.config();     // or start from a preset
```

The analysis keeps a copy of its `Config`. To change a setting later, build a new `Analysis` and `Solver`.

ENUM	MEMBERS
`OrderingMethod`	`Scalpel`
`Preset`	`Balanced`, `Sweep`, `OneShot`, `StructuredGrid`, `Careful` (Section 5.5)

5.4 CONFIGURING IN PYTHON

Pass settings as keyword arguments to `sparta.analyze`:

```
solver = sparta.analyze(A, ordering=sparta.Ordering.SCALPEL, nemin=32)
```

To keep a whole set of settings together, use `sparta.Config`. It is immutable: `with_` returns a changed copy.

```
config = sparta.Config.preset("sweep").with_(nemin=32)
solver = sparta.analyze(A, config=config)
```

CALL	RETURNS
`sparta.Config.defaults()`	The default settings, read from the compiled library.
`sparta.Config.preset(name)`	A preset by name, such as `"sweep"`.
`config.with_(**changes)`	A copy with the named settings changed.
`config.to_dict()`	The settings as keyword arguments for `analyze`.
`sparta.presets()`	Every preset name with a one-line summary.

Which value wins. A keyword argument overrides the same setting in `config`. One exception: `max_front` has a default of its own, `None`, and passing it does not override the config. To remove a config's front limit, change the config itself:

```
config = config.with_(max_front=None)
```

In a `Config`, `ordering` is stored as text (`"scalpel"`), so use the enum's `.value` there, and the enum itself as a keyword argument.

Changing settings later. The solve settings, `force_cpu` and `hybrid` can be passed to `factorize` to change them for an existing solver; anything left out keeps its current value. Everything else needs a new `analyze`.

```
solver.factorize(refinement_rtol=1e-10)   # tighter refinement from now on
```

5.4.1 PYTHON ENUMS

ENUM	MEMBERS	MEANING
`Ordering`	`SCALPEL`	Which ordering to use. SCALPEL is the only one.
`Path`	`AUTO`, `LDLT`, `LU`	Which method to use. `AUTO` decides from the matrix type.
`Backend`	`GPU`, `CPU`	Where a tuned configuration runs (Section 5.6).
`MatrixType`	`AUTO`	Look at the matrix and decide: LDL$^\intercal$ if symmetric, else LU.
	`REAL_SYMMETRIC`, `COMPLEX_SYMMETRIC`	$A = A^T$. Uses LDL$^\intercal$.
	`REAL_UNSYMMETRIC`, `COMPLEX_UNSYMMETRIC`	Any square matrix. Uses LU.
	`REAL_STRUCTURALLY_SYMMETRIC`, `COMPLEX_STRUCTURALLY_SYMMETRIC`	Symmetric pattern, unsymmetric values. Uses LU.
	`HERMITIAN`	A equals its conjugate transpose. Uses LU.
`RefinementStatus`	`CONVERGED`, `STAGNATED`, `ITERATION_LIMIT`, `NONFINITE`	How refinement ended (Section 6).

The enums also contain members reserved for later versions. Selecting one raises `NotImplementedError` rather than silently doing something else.

5.5 PRESETS

A preset is a named starting point for a kind of workload. It is not a guarantee: the best settings depend on your matrices, so measure before you keep a change.

RUST	PYTHON	USE IT FOR, AND WHAT IT CHANGES
`Balanced`	`"balanced"`	The defaults. One factorization of an unstructured mesh.
`Sweep`	`"sweep"`	One pattern factored many times. `wide_cycles = 6` and `separator_trials = 8`: more ordering work, paid once.
`OneShot`	`"one-shot"`	Factor once and discard. `wide_cycles = 0` and `separator_trials = 1`: a fast ordering with more fill-in.
`StructuredGrid`	`"structured-grid"`	Regular grids. `nemin = 32`: wider fronts.
`Careful`	`"careful"`	Ill-conditioned systems. Strict LU pivoting (`lu_diag_threshold = 0`) and `imbalance = 0.6`: a more balanced split.

5.6 TUNING TO YOUR COMPUTER

The defaults were measured on one machine, and presets are educated guesses. `sparta.optimize` measures instead. It factors your matrices under many candidate settings, keeps whatever is clearly faster and no less accurate, and saves the result under a name you choose.

```
result = sparta.optimize([A1, (A2, b2)], name="antenna")
solver = sparta.analyze(A, config="antenna")      # any later session
```

Pass matrices like the ones you will factor, at representative sizes: the best settings follow the matrices as much as the hardware. A problem is a sparse matrix, or a `(matrix, rhs)` pair with a vector or an `(n, k)` block. Without problems, `sparta.optimize()` tunes on two standard finite-element Helmholtz systems of 14 and 47 thousand unknowns, which is a reasonable general tuning for the computer.

What it prints. Every measurement prints one line as it finishes, with the count done, the total, the time spent and an estimate of the time left:

```
[ 5/78 0:06 | ~1:29 left] GPU nemin=16                  99.1 ms  (current)
[ 6/78 0:07 | ~1:29 left] GPU nemin=8                  101.8 ms  (+2.7% vs current)
...
SPARTA optimize: 2 problem(s) on Apple M4 Pro, 1:27
             default       tuned     gain
  GPU      100.4 ms     97.9 ms    1.03x
  CPU      338.7 ms    330.7 ms    1.02x
  Result: uses the GPU, 1.03x the speed of the default configuration (2% less time).
  Changed: narrow_cols 32 -> 16
```

The summary compares the default and the tuned settings, measured back to back at the end. Ctrl-C stops the search.

ARGUMENT	DEFAULT	WHAT IT DOES
`problems`	standard set	The matrices to tune on.
`name`	none	Save the result under this name. Preset names are refused.
`gpu`, `cpu`	`True`, `True`	Which backends to tune. With both, the result uses the faster one, and each keeps its own tuned settings, so `factorize(force_cpu=...)` can still switch.
`start`	defaults	A `Config`, profile or preset to start from and compare against.
`analysis`	`False`	Count analysis time too. Turn on when each pattern is factored only once.
`time_limit`	none	Seconds after which the search stops and keeps the best so far.
`iterations`	`1`	How many full passes over all settings to make. Each pass starts again from the settings the last one found (Section 5.6.1).
`min_gain`	`0.03`	The smallest improvement worth keeping, as a fraction. `0.03` keeps a change only when it saves more than 3% of the time.
`verbose`	`True`	Print progress and the summary.
`progress`	none	A function called with each progress report, for your own display.
`mtype`, `path`, `triangle`	as `analyze`	How to read the matrices.

How it searches. One setting at a time: first the ordering and front settings, on whichever backend is faster, then each backend's own settings. Each candidate is timed as the median of several factorize-and-solve runs. `lu_diag_threshold` is never changed, because it trades accuracy for speed. A full run on the standard problems takes one to two minutes on an M4 Pro.

What counts as faster. Timings vary a little from run to run, and a search that follows every small difference ends up following noise. A change is therefore kept only when all of these hold:

1. It saves more than `min_gain` of the time, 3% by default.
2. It saves more than the timing noise, measured as the spread of that candidate's own runs. On a small matrix that factors in a few milliseconds, the noise can be 10% or more, and then the bar is that high.
3. It is still faster when the old and new values are timed again, head to head. A candidate that was only lucky once prints `was not faster on a recheck` and is dropped.
4. Its residual is at most ten times the default settings' residual.

Raise `min_gain` to change only what clearly matters, for example `min_gain=0.1` for 10%.

The result, `sparta.Optimization`, has `config`, `backend`, `speedup`, `seconds` (default and tuned, per backend), `changes`, `iterations` and every `trials` entry. `print(result)` gives the summary.

5.6.1 ITERATIONS

Settings affect each other. A wider `nemin` changes which fronts exist, and so which kernel geometry suits them. A single pass tunes each setting while the ones after it still have their old values. With `iterations=3` the search makes up to three passes, each starting from what the last one found, so every setting is tuned again once the others have moved.

```
result = sparta.optimize([A1, A2], name="antenna", iterations=3)
```

A pass that changes nothing means the settings have settled, and the search stops there without spending time on the remaining passes:

```
[ 66/286 0:15 | ~0:48 left] iteration 2 of 3: starting again from the settings found
so far
[126/130 0:28 | ~0:01 left] iteration 2 changed nothing: the settings have settled,
skipping the rest
...
  Iterations: 2 of 3, stopped early.
```

So a large value costs nothing once the settings have settled. Each pass takes about as long as the first. The progress total and the time left count all requested passes, and drop when the search stops early. `result.iterations` gives `(run, asked)`.

5.6.2 SAVED PROFILES

A profile is a saved `Config`. All profiles live in one file, `configs.json`, in the SPARTA folder of your home directory: `~/Library/SPARTA` on macOS, `~/.sparta` elsewhere. Set `SPARTA_CONFIG` to use another file.

CALL	WHAT IT DOES
`sparta.analyze(A, config="name")`	Use a saved profile, or a preset of that name.
`sparta.Config.load(name)`	The saved profile as a `Config`.
`config.save(name, note="")`	Save any `Config` as a profile.

CALL	WHAT IT DOES
`sparta.Config.delete(name)`	Remove a profile.
`sparta.profiles()`	Every profile with a one-line summary: chip, backend, gain and date.
`sparta.Config.store()`	The path of `configs.json`.

A profile records the chip it was tuned on. Loading it on a different chip works, with a warning to tune again. A profile saved by an older SPARTA loads too: settings added since then take their defaults.

In Rust, `sparta::optimize::optimize` runs the same search, and `sparta::profiles` reads and writes the same file:

```rust
use sparta::optimize::{optimize, Options, Problem};

let problems = vec![Problem::ldlt(n, ptr, rows, values)?];
let options = Options { iterations: 3, ..Options::default() };
let outcome = optimize(&problems, &options, &mut |p| {
    eprintln!("{p}");   // one progress line
    true               // false stops the search
})?;
println!("{outcome}");
sparta::profiles::save("antenna", &outcome.profile())?;
let config = sparta::profiles::load("antenna")?.config;
```

`cargo run --release --example optimize -- NAME` tunes on the standard problems and saves the result as `NAME`.

5.7 ALL SETTINGS

Types are given in a language-neutral form. `uint32` is `u32` in Rust and `int` in Python; `uint64` is `usize` and `int`; `float64` is `f64` and `float`; "optional" is `Option` in Rust and `None` in Python.

NAME	TYPE	DEFAULT	WHAT IT IS FOR
Solve settings (Python only)			
`refinement_steps`	int	`-1`	Maximum refinement corrections per right-hand side.
`refinement_rtol`	float64	`1e-6`	Residual at which refinement stops.
`refinement_minimize`	bool	`True`	Scale each correction to shrink the residual most.
`equilibrate`	bool	`False`	Balance row and column sizes before converting to single precision.
`safe_mode`	bool	`False`	Re-check every new matrix passed to `factorize`.
`verbose`	bool	`False`	Print a short summary per solve.
`mtype`, `path`, `triangle`	enum, enum, text	automatic, `"full"`	How to read and factor the matrix.
Ordering			
`ordering`	enum	SCALPEL	The nested dissection. SCALPEL is the only one.

NAME	TYPE	DEFAULT	WHAT IT IS FOR
`imbalance`	float64	`0.8`	SCALPEL only. How unequal a split may be.
`wide_cycles`	uint64	`4`	SCALPEL only. Separator refinement cycles per split.
`fm_passes`	uint64	`2`	SCALPEL only. Refinement passes per separator.
`separator_trials`	uint64	`4`	SCALPEL only. Candidate separators tried per split.
Fronts			
`nemin`	uint64	`16`	How eagerly columns merge into wider fronts.
`max_front`	uint64, optional	unlimited	Largest front allowed, in rows.
`small_front_rows`	uint32	`32`	Front height where the GPU leaves its serial kernel.
Numerics			
`force_cpu`	bool	`False`	Double precision on the CPU instead of the GPU.
`hybrid_cpu`	float64	`0.0`	Share of the work the CPU does alongside the GPU (Metal only).
`lu_diag_threshold`	float64	`0.0`	LU only. How strongly to prefer the diagonal as pivot.
CPU backend			
`cpu_panel`	uint64	`32`	Pivots per panel before an update.
`cpu_block_rows`	uint64	`96`	Rows per block in the update.
`cpu_blas_min_rows`	uint64	`64`	Smallest block handed to BLAS.
GPU scheduling			
`schur_batch`	uint32	`0`	Panels completed between update rounds.
`panel_threads`	uint32	`512`	Threads per group in the panel factorization.
`solve_threads`	uint32	`0`	Threads per front in the solves.
GPU kernel geometry			
`panel`	uint32	`16`	Pivots per panel.
`schur_block`	uint32	`4`	Tile size in the main update.
`strip_block`	uint32	`4`	Tile height in the strip update.
`narrow_cols`	uint32	`32`	Width where a front takes the wide kernels.
`pivot_threads`	uint32	`128`	Threads per pivot search.
`schur_chunk`	uint32	`16`	Columns staged per update pass.

The sections below explain each group, with the allowed values.

5.8 SOLVE SETTINGS (PYTHON ONLY)

These control refinement and input checking, which live in the Python layer. Rust has no equivalent yet; Section 6 shows how to refine from Rust yourself.

SETTING	VALUES	WHAT IT DOES
`refinement_steps`	`-1`, `0`, or a positive integer	`-1` refines until `refinement_rtol` is reached or refinement stops improving, with a safety cap of 50. `0` switches refinement off. A positive number is a hard cap.
`refinement_rtol`	between 0 and 1	The relative residual $\|b - Ax\|/\|b\|$ at which refinement stops.
`refinement_minimize`	bool	Scales each correction so it shrinks the residual as much as possible. Costs one extra matrix-vector product per step on the CPU, and roughly halves the steps on badly conditioned matrices.
`equilibrate`	bool	Scales rows and columns to similar sizes before the values are rounded to single precision. Can help accuracy when entries differ by many orders of magnitude; costs extra memory and CPU work.
`safe_mode`	bool	Re-checks every new matrix passed to `factorize`. Off, a new matrix stored exactly like the analyzed one is read straight into the analyzed layout, which is much faster, and its values are not checked for infinities or NaNs. Either way a different pattern is detected and analyzed again.
`verbose`	bool	Prints a short summary per solve: method, ordering, fill-in, memory, timings and residuals. Adds the cost of measuring the residuals.

5.9 ORDERING SETTINGS

The ordering decides the fill-in, and so most of the memory and time. SCALPEL is SPARTA's own nested dissection: it runs in parallel and can be tuned. A slower, better ordering pays off when one pattern is factored many times, as in a frequency sweep. Measure the whole workload, because a faster ordering does not always give a faster factorization.

SETTING	VALUES	WHAT IT DOES
`imbalance`	0.5 to 0.9	The largest share of the matrix allowed in one half of a split. Near 0.5 asks for even halves; larger values give more freedom to find a small separator, which can reduce fill-in.
`wide_cycles`	0 or more	Cycles that widen a separator and search for a smaller replacement. `0` orders faster with more fill-in.
`fm_passes`	0 or more	Fiduccia–Mattheyses passes, a refinement that may accept temporary worsening to find a better separator. `0` is much faster and much worse.
`separator_trials`	1 or more	Starting separators tried per split; the best is kept. More trials help on irregular meshes.

`RAYON_NUM_THREADS` limits how many CPU threads SCALPEL and the rest of the analysis use.

More SCALPEL options, Rust only. `config.scalpel` has further fields for ordering experiments. Leave them at their defaults unless you are researching orderings.

FIELD	TYPE	DEFAULT	WHAT IT DOES
`leaf_size`	`usize`	`32`	Parts at most this size (1 to 256) are ordered by exact minimum degree instead of being cut further.
`parallel_threshold`	`usize`	`4096`	Parts at least this size may be processed in parallel.
`spectral_candidates`	`usize`	`4`	Extra starting separators cut across the graph's smoothest vibration modes. They give flatter cuts and a shallower tree, which factors and solves faster on the GPU. `0` orders faster.
`refine_all_candidates`	`bool`	`true`	Refine every candidate separator before choosing one.
`half_levels`	`bool`	`true`	Build starting separators from distances to two ends of the graph.
`balance_quality`	`bool`	`true`	Favour balanced halves as well as small separators.
`refinement_passes`	`usize`	`4`	Greedy improvement sweeps per level. `0` skips them.
`one_sided_refinement`	`bool`	`false`	Move vertices only towards the smaller half.
`coarsen_threshold`	`usize`	`512`	Parts above this size are simplified before cutting, then refined. `0` disables; otherwise at least 32.
`compress`	`bool`	`true`	Merge vertices with identical neighbours before ordering.
`cover_refinement`	`bool`	`true`	Improve separators with a vertex-cover search at their boundaries.

5.10 FRONT SETTINGS

SETTING	VALUES	WHAT IT DOES
`nemin`	1 or more	The fewest columns a supernode must have before it may merge with its neighbour. Larger values give fewer, wider fronts: better GPU efficiency, but more fill-in and memory. Try 32 when many small fronts dominate.
`max_front`	1 or more, or unlimited	The largest front allowed, in rows. Analysis fails if a front would be larger, before any GPU memory is claimed. Use it to reject a pathological matrix early; it is not a total memory budget.
`small_front_rows`	1 or more	Fronts up to this height use a serial GPU kernel; taller ones use cooperative kernels. The faster choice depends on the front sizes.

5.11 NUMERICAL SETTINGS

SETTING	VALUES	WHAT IT DOES
`force_cpu`	bool	Factor and solve in double precision on the CPU instead of the GPU. Much slower, but accurate to about sixteen digits. See Section 6.
`hybrid_cpu`	0 to below 1	Share of the factorization work the CPU does while the GPU does the rest. `0` is the GPU alone. A starting point: the share adapts after every

SETTING	VALUES	WHAT IT DOES
		factorization. Python also takes `hybrid=True`, which is 30%. Ignored except on Metal. See Section 6.7.
`lu_diag_threshold`	0 to 1	LU only. At `0`, each pivot is the largest candidate in its column (strict partial pivoting). Above 0, the diagonal entry is kept as pivot if it is at least this fraction of the largest candidate: `0.1` accepts a diagonal one tenth as large. This saves row swaps but accepts weaker pivots, and **changes the results**. Check residuals if you use it.

5.12 CPU BACKEND SETTINGS

These only matter with `force_cpu`.

SETTING	VALUES	WHAT IT DOES
`cpu_panel`	1 or more	Pivots gathered before a trailing update, on the LU path and on fronts LAPACK cannot factor.
`cpu_block_rows`	1 or more	Rows per block in the trailing update, and the unit of work when one front is split across threads. Smaller blocks waste less work but add overhead.
`cpu_blas_min_rows`	1 or more	The smallest trailing block handed to BLAS. Raise it to avoid BLAS overhead on small blocks.

5.13 GPU SETTINGS (ADVANCED)

The defaults are measured on Apple Silicon. Leave these alone unless you are tuning for different hardware, and measure every change.

SETTING	VALUES	WHAT IT DOES
`schur_batch`	0 or more	How many pivot panels complete between update rounds for the rest of the front. `0` runs one round per level of the tree.
`panel_threads`	`0`, or 32 to 1024	Threads per group in the panel factorization. `0` means the built-in 512.
`solve_threads`	`0`, or 32 to 1024	Threads per front in the solves. `0` chooses per front. Values up to 256 select the 256-thread path, larger values the 512-thread path.
`panel`	16	Pivots per panel. Other widths need kernel changes.
`schur_block`	1 to 8	Tile size in the main update: each group computes a `16 × schur_block` square tile.
`strip_block`	1 to 8	Tile height in the strip update: `16 × strip_block` rows, which must be a multiple of `panel`.
`narrow_cols`	1 or more	Fronts wider than this take the wide-front kernels.
`pivot_threads`	multiple of 32, up to 1024	Threads cooperating on one pivot search.
`schur_chunk`	1 or more	Columns staged together per update pass.

The last six settings are compiled into the GPU kernels. A combination the process has not used before costs about 300 ms of compilation the first time a solver is created. Both update kernels must also fit the GPU's 32 KiB of shared memory per group:

- main update: $2 \times (16 \times \text{schur_block}) \times \text{schur_chunk} \times 8$ bytes,
- strip update: $(16 \times \text{strip_block} + 16) \times \text{schur_chunk} \times 8$ bytes.

A combination that breaks a limit is rejected with a message naming the setting.

5.14 ENVIRONMENT VARIABLES

VARIABLE	EFFECT
`SPARTA_SCHUR_BATCH`	Overrides the default of `schur_batch`.
`SPARTA_PANEL_THREADS`	Overrides the default of `panel_threads`.
`SPARTA_SOLVE_THREADS`	Overrides the default of `solve_threads`.
`SPARTA_LU_DIAG_THRESHOLD`	Overrides the default of `lu_diag_threshold`.
`RAYON_NUM_THREADS`	Limits the CPU threads used by the analysis. Set it before the program starts.
`SPARTA_PROFILE`	`1` prints the time of each stage of analysis and factorization; `2` adds GPU dispatch detail.

Settings you assign explicitly in code win over these variables.

When comparing settings, measure analysis, factorization and solve time, and the factor memory, over your whole workload. In Rust, `Analysis` reports `max_front_size()` and `factor_bytes()`, and `Solver` reports `factor_seconds()`. In Python, `solver.info` and `solver.diagnostics()` give the same.

CHAPTER 6

PRECISION

This chapter explains why a solve is never exact, how to tell how good an answer is, and exactly what to do when it is not good enough.

6.1 WHY ANSWERS ARE NOT EXACT

Rounding. The GPU computes in single precision, which keeps about seven significant digits. Every operation rounds, and a factorization performs billions of them. The CPU backend uses double precision, about sixteen digits.

Conditioning. Some systems amplify small errors. The **condition number** of A measures by how much: with a condition number of 10^6, rounding errors in the seventh digit can grow into errors in the first. Simulation matrices at low frequencies, or with very fine meshes, are often badly conditioned.

Residual and error are different. The **relative residual** $\|b - Ax\|/\|b\|$ measures how well x satisfies the equations. It is cheap to compute. The **error** $\|x - x_{\text{true}}\|/\|x_{\text{true}}\|$ is what you care about, but it cannot be computed without knowing the true answer. On a badly conditioned matrix the error can be far larger than the residual. On one measured test system (condition number about 2.4×10^9), a residual of 4×10^{-7} came with an error of 8×10^{-5}, two hundred times larger.

> NOTE
>
> **Ask for a tighter residual than the accuracy you need.** On that system, an error of 2×10^{-8} needed `refinement_rtol = 1e-10`.

6.2 THE OPTIONS AT A GLANCE

APPROACH	ACCURACY	COST	AVAILABLE IN
GPU, no refinement	residual about 10^{-5}	fastest	Rust, Python
GPU with refinement	residual 10^{-6} by default, or as tight as you ask	about 9% more time on large systems at the default	Python built in; Rust with Section 6.5.2
CPU backend	double precision, about 16 digits	much slower	Rust, Python

Iterative refinement is the usual answer. It keeps the fast single-precision factors, and repeats three steps: compute the residual $r = b - Ax$ in double precision, solve $Ad = r$ with the single-precision factors, and add the correction d to x. Each round gains several digits, until the residual reaches the target.

6.3 WHAT TO DO: STEP BY STEP

1. **Keep your data in double precision.** Pass A and b as float64 if they are real, complex128 if they are complex. Refinement measures the residual against the values you pass, so it cannot recover digits that were lost before SPARTA saw them.
2. **Check the factorization** before trusting a solve (Section 6.4).
3. **Use refinement,** and set the residual target tighter than the accuracy you need (Section 6.5).
4. **Read how refinement ended.** If it converged, you are done. If it stagnated, continue with step 5.
5. **Help the factorization:** turn on `equilibrate`, use the `careful` preset, and make sure `lu_diag_threshold` is `0`.
6. **If that is still not enough, switch to the CPU backend** (Section 6.6). It is slower, but computes in double precision throughout.
7. **Verify the final answer** by computing its residual yourself.

6.4 CHECKING THE FACTORIZATION

Python. Right after `factorize()`, before any solve, `solver.quality` describes how close the factorization came to breaking down. It costs no extra GPU work.

```
solver.factorize()
q = solver.quality
print(q.verdict)       # 'good', 'degraded', 'unusable' or 'unknown'
print(q.usable)        # False only when the verdict is 'unusable'
```

VERDICT	MEANING AND WHAT TO DO
`good`	The factors are sound. Solve normally.
`degraded`	Some pivots are close to single-precision resolution. Solves are still usable, but rely on refinement and check the result.
`unusable`	The factorization effectively broke down. Refinement will not recover it: use the CPU backend.
`unknown`	There is not enough information to judge, which happens on the CPU backend before any refined solve. Check the refinement report or the residual instead.

Rust. Two methods give the raw signals:

```
let (smallest, largest) = solver.pivot_range().unwrap();
let nudged = solver.perturbed_pivots();
```

- `pivot_range()` returns the smallest and largest pivot size. A ratio `smallest / largest` near single-precision resolution (about 10^{-7}) is a warning; far below it, expect the solve to fail.
- `perturbed_pivots()` counts pivots SPARTA had to nudge away from zero to avoid a breakdown. Nonzero means the factors belong to a slightly different matrix: use refinement.

Neither a good verdict nor zero perturbed pivots guarantees an accurate answer. They catch breakdowns; the residual confirms the result.

6.5 GETTING MORE ACCURACY FROM THE GPU

6.5.1 IN PYTHON

Refinement is on by default, with a target residual of 10^{-6}:

```python
# Keep double precision: float64 for a real system ...
A = A.astype(np.float64)
b = b.astype(np.float64)
# ... or complex128 for a complex one.
A = A.astype(np.complex128)
b = b.astype(np.complex128)

# A tighter target than the default 1e-6.
solver = sparta.analyze(A, refinement_rtol=1e-10)
solver.factorize()
x = solver.solve(b)

# One report per right-hand side.
for report in solver.refinement_reports:
    print(report.status, report.corrections,
          report.relative_residual, report.estimated_relative_error)
```

A refined solve returns double precision whatever you passed in: float64 when the matrix and b are both real, complex128 otherwise. Each report tells you:

FIELD	MEANING
`status`	`CONVERGED`: the target was reached. `STAGNATED`: corrections stopped helping; the best answer found is returned. `ITERATION_LIMIT`: `refinement_steps` ran out. `NONFINITE`: the solve produced infinities or NaNs.
`corrections`	How many corrections were applied.
`relative_residual`	The final relative residual.
`estimated_relative_error`	An estimate of the error in x itself, from the size of the last correction. Usually the number you actually care about. It is generous while refinement converges, and unreliable once it has stagnated.
`history`	The residual after each step.

Other refinement controls:

```python
# At most five corrections.
sparta.analyze(A, refinement_steps=5)
# Refinement off: the bare single-precision solve.
sparta.analyze(A, refinement_steps=0)
# Rescale rows and columns before rounding to single precision.
sparta.analyze(A, equilibrate=True)
# Change the target for an existing solver.
solver.factorize(refinement_rtol=1e-12)
```

6.5.2 IN RUST

Rust solves return the single-precision answer; refinement is up to you. The function below implements it. It needs a way to multiply your matrix by a vector in double precision, passed as `matvec`.

```rust
use sparta::{Complex32, Complex64, Solver};

/// Solves A x = b, then corrects x in double precision until
/// ||b - A x|| / ||b|| <= rtol or max_steps corrections have been made.
fn solve_refined(
    solver: &mut Solver,
    matvec: impl Fn(&[Complex64]) -> Vec<Complex64>,
    b: &[Complex64],
    rtol: f64,
    max_steps: usize,
) -> sparta::Result<Vec<Complex64>> {
    let norm = |v: &[Complex64]| v.iter().map(|z| z.norm_sqr()).sum::<f64>().sqrt();
    let target = rtol * norm(b);
    let mut x = vec![Complex64::new(0.0, 0.0); b.len()];
    let mut r = b.to_vec(); // residual b - A x, in double precision
    for _ in 0..=max_steps {
        let scale = norm(&r);
        if scale <= target {
            break;
        }
        // Scale to unit size before rounding, so small residuals keep their digits.
        let rhs: Vec<Complex32> = r
            .iter()
            .map(|v| Complex32::new((v.re / scale) as f32, (v.im / scale) as f32))
            .collect();
        let d = solver.solve(&rhs)?;
        for (xi, di) in x.iter_mut().zip(&d) {
            *xi += Complex64::new(di.re as f64, di.im as f64) * scale;
        }
        let ax = matvec(&x);
        for ((ri, bi), axi) in r.iter_mut().zip(b).zip(&ax) {
            *ri = bi - axi;
        }
    }
    Ok(x)
}
```

The function is written for a complex matrix. For a real one, the same steps work with `f64` in place of `Complex64`, `f32` in place of `Complex32`, and `solver.solve_values(&rhs, 1)` in place of `solver.solve(&rhs)`.

Call it after `factor`:

```rust
solver.factor(&values)?;
let x = solve_refined(&mut solver, |x| my_matrix.multiply(x), &b, 1e-10, 20)?;
```

The residual must be computed from your original double-precision matrix, not from the single-precision values you passed to `factor`: that is where the extra digits come from. If the residual stops shrinking, stop refining and move to the CPU backend.

6.6 SOLVING ON THE CPU IN DOUBLE PRECISION

`force_cpu` performs the factorization and the solves in double precision on the CPU, with the same analysis. It is much slower than the GPU but keeps about sixteen digits throughout, which is the answer for systems too badly conditioned for refinement. SPARTA never switches to the CPU by itself: a GPU failure is reported as an error.

Python

```
solver = sparta.analyze(A, force_cpu=True, refinement_steps=0)
solver.factorize()
x = solver.solve(b)
solver.compute_dtype   # float64 for a real A, complex128 for a complex one

# Or switch an existing solver, keeping its analysis, and back again.
solver.factorize(force_cpu=True)
solver.factorize(force_cpu=False)
```

Refinement also works on the CPU backend, but is rarely needed there.

Rust. Set `force_cpu` and pass double-precision values, `f64` or `Complex64`:

```
let config = Config { force_cpu: true, ..Config::default() };
let mut solver = Solver::new(Analysis::ldlt(n, &col_ptr, &row_idx, &config)?)?;
// values and b are &[f64] or &[Complex64]; the last argument counts the b's.
solver.factor_values(&values)?;
let x = solver.solve_values(&b, 1)?;
```

On Linux and Windows, install a BLAS library first, or the CPU backend runs much more slowly (Section 2.6).

6.7 CPU AND GPU TOGETHER

On a Mac the CPU and the GPU share one memory, so they can factor one matrix together. With `hybrid` on, the CPU takes whole branches of the elimination tree while the GPU factors the rest, and the GPU finishes from where the two meet. The CPU computes in the GPU's precision, so the answers match a GPU-only factorization to single-precision rounding. It works for LDL^T and LU, real and complex.

```
solver = sparta.analyze(A, hybrid=True).factorize()   # the CPU starts at 30% of the
work
solver.factorize(A2, hybrid=0.2)                      # or start it at 20%
solver.factorize(A3, hybrid=False)                    # the GPU alone again
solver.hybrid_timing                                  # how the last factorization
split
```

In Rust set `hybrid_cpu` in the `Config`; in C set `config.hybrid_cpu` before `sparta_analyze`; in Julia pass `hybrid = true`.

- **When it starts.** The first factorization already runs split. After that the split adapts to the measured speed of each device, so a sweep settles from the third factorization on. Changing `hybrid` in `factorize` rebuilds the GPU factors.
- **Where it applies.** Only the Metal backend has a hybrid path. With `force_cpu`, on Vulkan, and without a GPU the setting is ignored.
- **What it gains.** On an M4 Pro with `RAYON_NUM_THREADS=10` (the performance cores): 1.37× on an 887k-unknown LDL^T system (1.49 s to 1.10 s), 1.25× at 457k, 1.10× at 109k. LU gains less, about 1.07× at 887k, because the CPU half of LU is slower relative to the GPU.

6.8 WHEN REFINEMENT DOES NOT CONVERGE

WHAT YOU SEE	WHAT TO DO
`status` is `STAGNATED` with a residual above the target	Try `equilibrate=True`. Then the `careful` preset. Then `force_cpu=True`.
`status` is `ITERATION_LIMIT`	Raise `refinement_steps`, or set it to `-1`.
`status` is `NONFINITE`, or `quality.verdict` is `unusable`	The single-precision factorization broke down. Use `force_cpu=True`.
Residual is small but the answer is still inaccurate	The matrix is badly conditioned. Tighten `refinement_rtol` by two or three orders of magnitude, and read `estimated_relative_error`.
`perturbed_pivots` is nonzero	Rely on refinement, and check the residual. Consider the CPU backend if refinement stagnates.
You set `lu_diag_threshold` above 0	Set it back to `0`: strict pivoting is the most stable.

6.9 CHECKING THE FINAL ANSWER

Whatever route you take, the residual in double precision is the final check:

```
residual = np.linalg.norm(b - A @ x) / np.linalg.norm(b)
```

In Rust, compute $\|b - Ax\|/\|b\|$ with your own double-precision matrix-vector product. A small residual together with a good `quality` verdict, or a converged refinement report with a small `estimated_relative_error`, is as much assurance as a solver can give.